Mobile Navigation and Localization

Zackory Erickson

Carnegie Mellon University
The Robotics Institute

16-762: Mobile Manipulation
Spring 2026

Overview

Navigation Stack

Probabilistic Localization

Monte Carlo Localization

Global and Local Planning

SLAM

Implementations on Stretch

The “Where” Problem: Localization and Mapping

Key Question: Where is the robot located within its environment?

1. **Mapping** (e.g. SLAM): The robot enters an unknown room and builds a floor plan (Map) using its sensors.

2. **Localization**: The robot is placed in a known room and must figure out its position (x, y) and orientation (θ) by matching what it “sees” to the Map.

3. **Pose Estimation**: Continuously tracking movement using sensors.

Challenges:

- Sensors are noisy (Lidar isn’t perfect).
- Environments look similar (one hallway looks like another).
- The “Kidnapped Robot”: What should the robot do if you pick it up and move it?

The “How” Problem: Planning and Control

- **Key Question**: How does the robot get from Start to Goal safely?
- **Global Planning**:
  - From a static map, find a valid path from the Living Room to the Kitchen.
- **Local Control**:
  - Following the Global Path while avoiding dynamic obstacles that aren’t on the map (people, pets, moved chairs).
- **Physical Constraints**:
  - Kinematics: The Stretch 3 is a Differential Drive robot. It cannot move sideways (strafe). It must rotate to face its direction of travel.
  - Footprint: The robot isn’t a point; it has width, a base, and an arm. Navigation must ensure no part of the robot hits an obstacle.

Stretch Hardware for Navigation

Key Sensors:

- **Lidar**: RPLidar A1 (2D laser scanner, ~12m range, 360° FOV minus mast occlusion).
- **RGB-D**: Intel RealSense D435i (Depth + IMU).
- **Proprioception**: High-resolution wheel encoders + 9-DoF IMU (in base and camera).

The Navigation Stack Architecture

- **Problem**: Moving from $q_{start} = (x, y, \theta)$ to q_{goal} while avoiding obstacles and minimizing cost.
- **Nav2** Approach (ROS 2):
  - **Map Server**: Provides the static environment.
  - **Localization** (Adaptive Monte Carlo Localization): Estimates pose map frame $(T_{map \to base_link})$.
  - **Global Planner**: Pathfinding on the static map.
  - **Local Planner** (Controller): Velocity command generation (v, ω) considering dynamics and dynamic obstacles.
  - **Recovery Behaviors**: Clearing costmaps, spinning, backing up.
  - **Behavior Trees**: Decision-making between these nodes.

Probabilistic Localization: The Bayes Filter

- **Belief**: $bel(x_t) = p(x_t \mid z_{1:t}, u_{1:t})$
  - x_t: State (Pose: x, y, θ).
  - z: Observations (Lidar scans).
  - u: Controls (Odometry / commanded velocity).
- Recursive Update Equation:

$$bel(x_t) = \eta \underbrace{p(z_t \mid x_t)}_{\text{Measurement Model}} \int \underbrace{p(x_t \mid u_t, x_{t-1})}_{\text{Motion Model}} bel(x_{t-1}) dx_{t-1}$$

- For a discrete space:

$$bel(x_t) = \eta \; p(z_t \mid x_t) \sum p(x_t \mid u_t, x_{t-1}) bel(x_{t-1})$$

η is the normalization constant, the denominator of the Bayes' rule. In practice, we often ignore this term.

Probabilistic Localization: The Bayes Filter

$$bel(x_t) = \eta \; p(z_t \,|\, x_t) \int p(x_t \,|\, u_t, x_{t-1}) bel(x_{t-1}) dx_{t-1}$$

1. Prediction Step: Project state forward using odometry (u_t). Increases uncertainty.

$$\bar{bel}(x_t) = \int p(x_t \,|\, u_t, x_{t-1}) bel(x_{t-1}) dx_{t-1}$$

2. Correction Step: Weight state by likelihood of sensor data (z_t). Decreases uncertainty.

$$bel(x_t) = p(z_t \,|\, x_t) \bar{bel}(x_t)$$

Monte Carlo Localization (MCL/AMCL)

- Representation: Represent the belief distribution $bel(x_t)$ as a set of M particles.

$$S_t = \{\langle x_t^{[m]}, w_t^{[m]} \rangle\}_{m=1}^{M}$$

- MCL algorithm:

  1. **Resampling**: Draw particles from S_{t-1} based on weights w_{t-1}.
  2. **Sampling** (Motion): Apply probabilistic motion model to each particle.
  3. **Importance Weighting** (Measurement): Assign weight $w_t^{[m]} = p(z_t \mid x_t^{[m]})$.

- **Adaptive MCL** (Kullback-Leibler distance sampling): Dynamically adjusts the number of particles (M) based on the complexity of the belief (e.g., fewer particles when position is certain, more when lost).

Costmaps: Representations for Planning

- Occupancy Grid: Static binary (Occupied, Free, Unknown).
- Layered Costmap:
  - **Static Layer**: Derived from the map server.
  - **Obstacle Layer**: Real-time updates from Lidar/Depth Camera (clears and marks).
  - **Inflation Layer**: Propagates costs outward from lethal obstacles.
- Cost Representation:
  - $C(x, y) = \infty$ if obstacle distance $d < R_{robot}$
  - $C(x, y) = \alpha \cdot e^{-\beta \cdot d}$ (Exponential decay) if $R_{robot} < d < R_{inflation}$
- Stretch 3: Due to the design of the base and telescoping arm, the *Footprint* is often defined as a circle for fast collision checking, or a polygon if high precision is needed in narrow corridors.

Global Path Planning

- **Objective**: Find a collision-free path in the configuration space.
- Common Algorithms:
  - Dijkstra: Guaranteed shortest path, examines all nodes.
  - A*: Heuristic-driven ($f(n) = g(n) + h(n)$). Faster.
- **Nav2 on Stretch**:
  - Stretch typically uses the **Smac Planner 2D** (A* implementation) or **NavFn** (A* or Dijkstra).
  - Utilizes cost-aware smoothing to ensure the path isn't jerky.
  - Output: A series of waypoints (x_i, y_i) in the Map frame.

Local Planning (Control)

- **Objective**: Follow the global path while avoiding dynamic obstacles and respecting kinematic constraints.
- **Question**: Could you use your IK implementation to move the base around a map?
- Differential Drive Kinematics (Stretch):
    - Non-holonomic constraint: $\dot{y} = 0$ (cannot slide sideways).
- **DWB** (Dynamic Window Approach):
    1. Sample pairs of (v, ω) reachable in the next time step.
    2. Simulate trajectories for a short horizon.
    3. Score trajectories: $J = \alpha \cdot \text{heading} + \beta \cdot \text{dist} + \gamma \cdot \text{vel}$

       heading = alignment with target goal | *dist* = distance to closest obstacle | *vel* = forward velocity
    4. Select trajectory with max J.
- **MPPI** (Model Predictive Path Integral): GPU-accelerated sampling of thousands of trajectories. Smoother handling of dynamic environments.

SLAM: Building the Map

- Chicken and Egg Problem: Need a map to localize; need location to map.
- Simultaneous Localization and Mapping (SLAM)
- **Graph-Based SLAM** (SLAM Toolbox / Karto SLAM):
    - Nodes: Robot poses.
    - Edges: Spatial constraints from odometry and scan matching.
    - Optimization: Minimize the error in the constraint graph (Loop Closure).

Mapping on Stretch 3

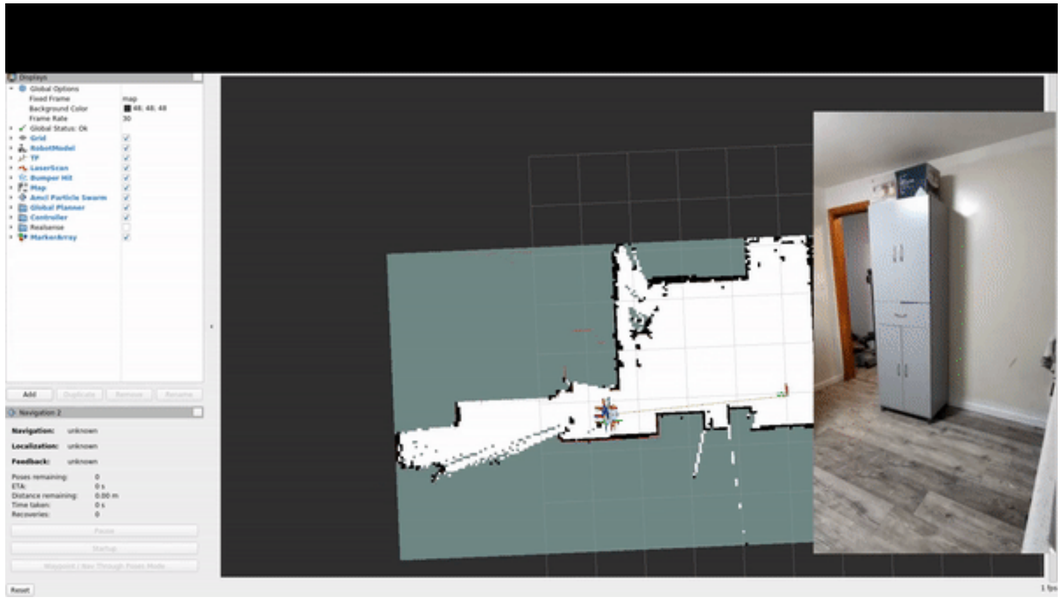
- Run:
  `ros2 launch stretch_nav2 offline_mapping.launch.py`

- Rviz will show the robot and the map that is being constructed.

- Use xbox controller or keyboard to teleoperate the robot around and cover the entire space. For keyboard: add `teleop_type:=keyboard`.

- Manual teleoperation to cover the area.

- Loop closure is critical: Return to previously visited areas to correct drift.



Mapping on Stretch 3

- Once you are happy with your map in Rviz, run the following commands in a new terminal to save the map.

```
mkdir ${HELLO_FLEET_PATH}/maps

ros2 run nav2_map_server map_saver_cli -f $
{HELLO_FLEET_PATH}/maps/<map_name>
```

- You can then view your new map with:

```
eog ${HELLO_FLEET_PATH}/maps/<map_name>.pgm
```

Navigation on Stretch 3

- **Run**:
  `ros2 launch stretch_nav2 navigation.launch.py map:=${HELLO_FLEET_PATH}/maps/<map_name>.yaml`

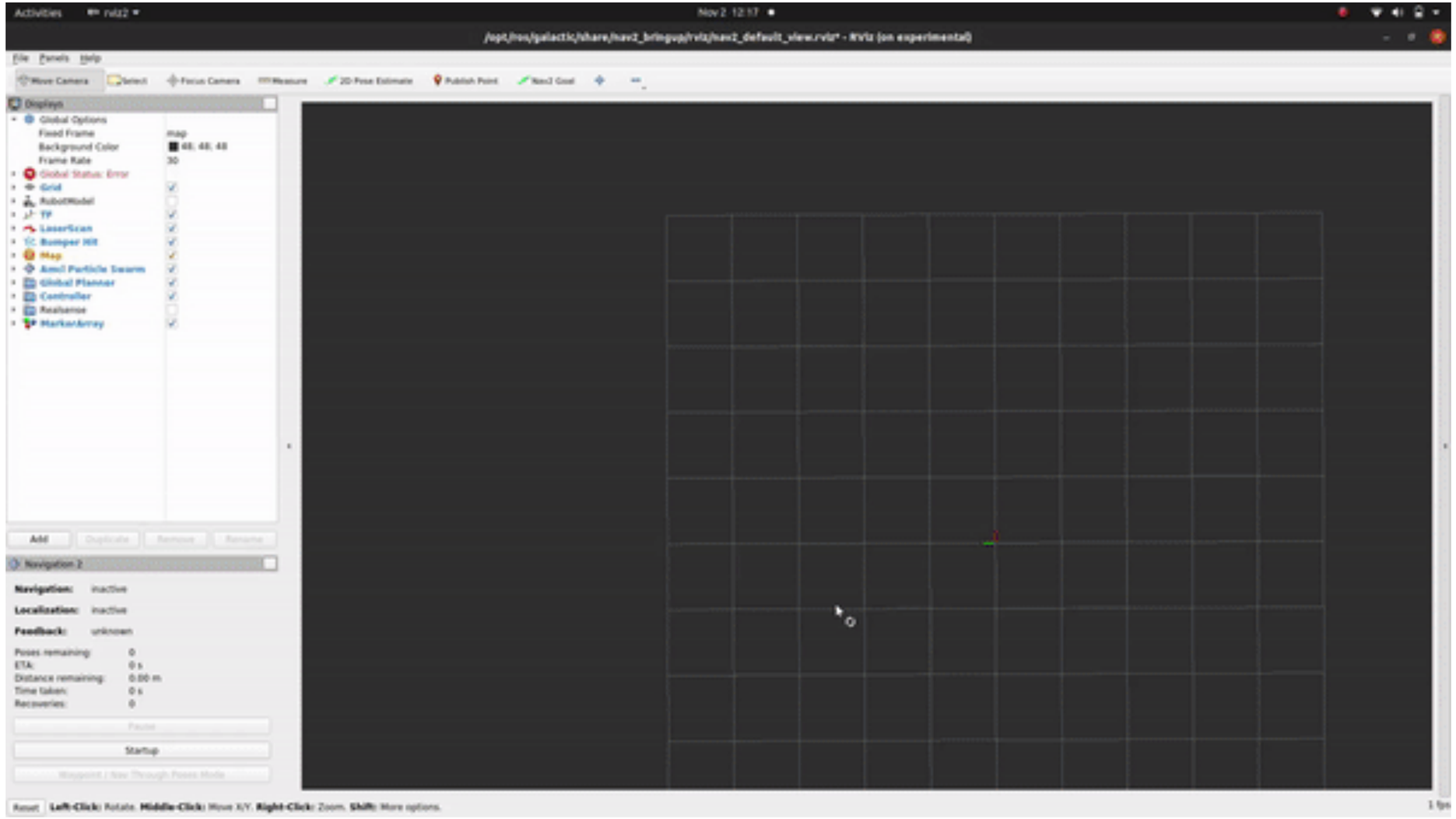
- RViz window pops up. Press the `Startup` button.

- **Note**: The robot doesn’t know where it is yet.

- To give an initial estimate to AMCL, press `2D Pose Estimate` from the top bar of Rviz to lay an arrow down roughly where the robot is located.

- Use `2D Nav Goal` to lay down an arrow where you'd like the robot to navigate.

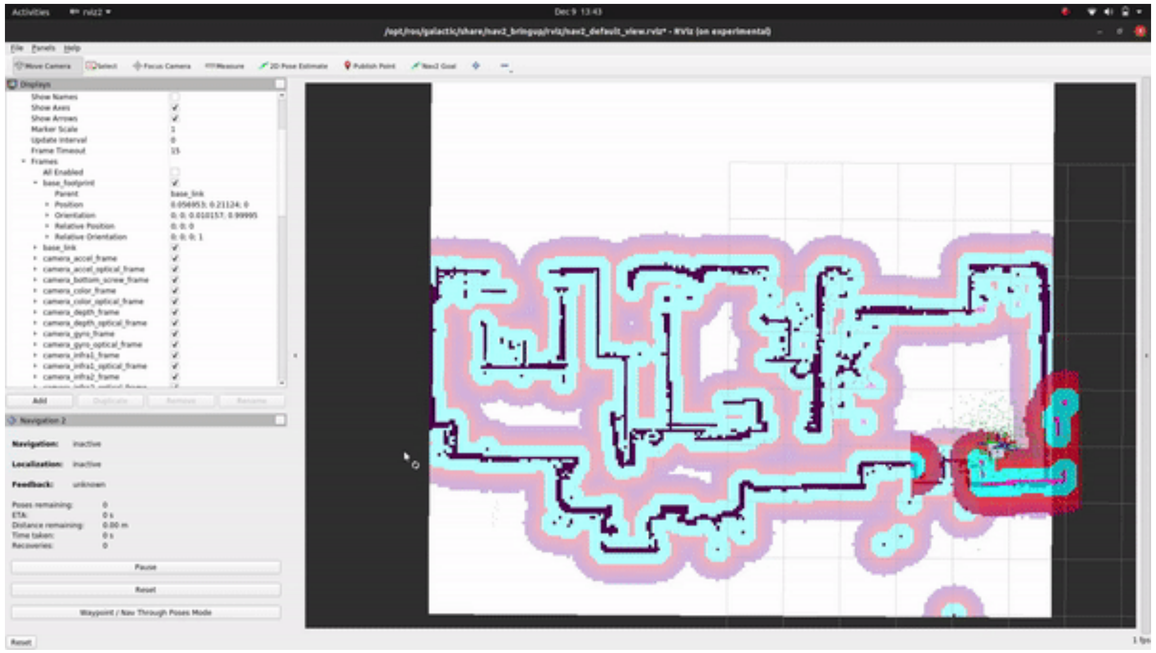
Navigation on Stretch 3



Autonomous Navigation in Python

- First, run:
  ```
  ros2 launch stretch_nav2 navigation.launch.py map:=$
  {HELLO_FLEET_PATH}/maps/<map_name>.yaml
  ```

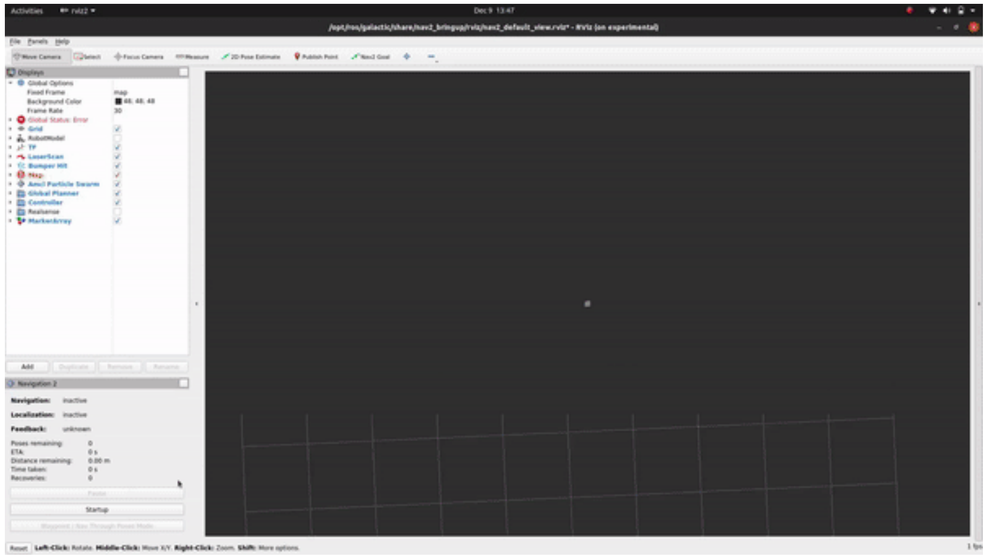
- Create arrows and save the (x, y) positions for where you want the robot to move between on your map.



Autonomous Navigation in Python

```
navigator = BasicNavigator()
route = [[0.0, 0.0], ...] # Put your (x, y) positions here
route_poses = []
pose = PoseStamped()
pose.header.frame_id = 'map'
pose.header.stamp = navigator.get_clock().now().to_msg()
pose.pose.orientation.w = 1.0
for pt in route[1:]:
    pose.pose.position.x = pt[0]
    pose.pose.position.y = pt[1]
    route_poses.append(deepcopy(pose))

nav_start = navigator.get_clock().now()
navigator.followWaypoints(route_poses)
```



Find example code here: https://github.com/hello-robot/stretch_ros2/blob/humble/stretch_nav2/stretch_nav2/simple_commander_demo.py#L30

Additional notes

- **Odometry quality**: Stretch's odometry is good but not perfect. Wheel slip on carpet is a major source of error. Fusion of IMU and wheel encoders can help (e.g. Extended Kalman Filter). Continuous localization with Lidar also helps.

- What happens if we pick up Stretch and move it? How does AMCL recover (or fail)?

- **Behavior Trees**: This is fairly standard. Navigation is not just a `while` loop; it is a tree of decisions (Recovery, Replanning, Context Switching).

Ongoing Research Topics

- **Semantic Navigation**: “Go to the kitchen” vs “Go to (x,y)”. General-purpose solution calls for combining Nav2/SLAM with Vision/LLMs.
- **Social Navigation**: Adjusting cost functions to avoid humans or navigate in dense human crowds comfortably.
- **Visual SLAM**: Using RGB-D for VSLAM when Lidar features are sparse (e.g., long hallways).